



CENTRO UNIVERSITÁRIO INTERNACIONAL UNINTER
ESCOLA SUPERIOR POLITÉCNICA
ANÁLISE E DESENVOLVIMENTO DE SISTEMAS - ADS
PROJETO MULTIDISCIPLINAR DESENVOLVIMENTO BACK-END

RODRIGO P ALCANTARA, 4844626
PROF. WINSTON SEN LUN FUNG

CAMPINAS-SP

2026

Sumário

1	INTRODUÇÃO	1
2	ANÁLISE DE REQUISITOS	1
2.1	REQUISITOS FUNCIONAIS (RF)	1
2.2	REQUISITOS NÃO FUNCIONAIS (RNF)	2
3	MODELAGEM E ARQUITETURA	3
3.1	DIAGRAMA DE CASOS DE USO.....	3
3.2	DIAGRAMA DE CLASSES	4
3.3	DIAGRAMA ENTIDADE-RELACIONAMENTO (DER)	5
4	IMPLEMENTAÇÃO	6
4.1	ESTRUTURA DE ROTAS DE ENDPOINTS (FAStPI).....	6
4.2	SEGURANÇA E PROTEÇÃO DE DADOS	8
4.2.1	<i>Controle de Acesso Baseado em Atribuições (RBAC):</i>	8
4.2.2	<i>Tratamento de Dados Pessoais Sensíveis:</i>	8
5	PLANO DE TESTES E QUALIDADE	9
5.1	CENÁRIOS DE TESTES FUNCIONAIS	9
5.2	CENÁRIOS DE TESTE NÃO FUNCIONAIS	10
5.3	EVIDÊNCIAS DE EXECUÇÃO (SWAGGER)	11
6	CONCLUSÃO.....	12
7	REFERÊNCIAS	12

1 INTRODUÇÃO

O presente projeto apresenta a proposta de desenvolvimento do Sistema de Gestão Hospitalar e de Serviços de Saúde (SGHSS) para a instituição VidaPlus, com foco exclusivo na arquitetura e implementação Back-end. O objetivo principal é conceber uma solução robusta e escalável que centralize a lógica de negócios para processos críticos, tais como: o cadastro de pacientes, o agendamento de consultas e o gerenciamento de prontuários eletrônicos.

Considerando a alta criticidade e a natureza sensível dos dados médicos, o desenvolvimento prioriza a criação de uma API REST eficiente, segura e em total conformidade com as diretrizes legais vigentes, garantindo a integridade, a confidencialidade e a alta disponibilidade das informações de saúde.

2 ANÁLISE DE REQUISITOS

Abaixo estão mapeados os requisitos funcionais e não funcionais do sistema. Conforme os critérios de qualidade de software exigidos, cada requisito possui sua respectiva prioridade técnica estipulada para a implantação adequada do ecossistema do back-end.

2.1 REQUISITOS FUNCIONAIS (RF)

ID	Descrição do Requisito	Prioridade
RF01	O sistema deve permitir o CRUD (Criar, Ler, Atualizar e Deletar) de dados dos pacientes.	Alta
RF02	O sistema deve permitir o cadastro, alteração e consulta de médicos e suas respectivas especialidades.	Alta
RF03	O sistema deve processar o agendamento, cancelamento e listagem de consultas médicas.	Alta
RF04	O sistema deve registrar e manter o histórico de prontuários médicos vinculados aos pacientes.	Alta

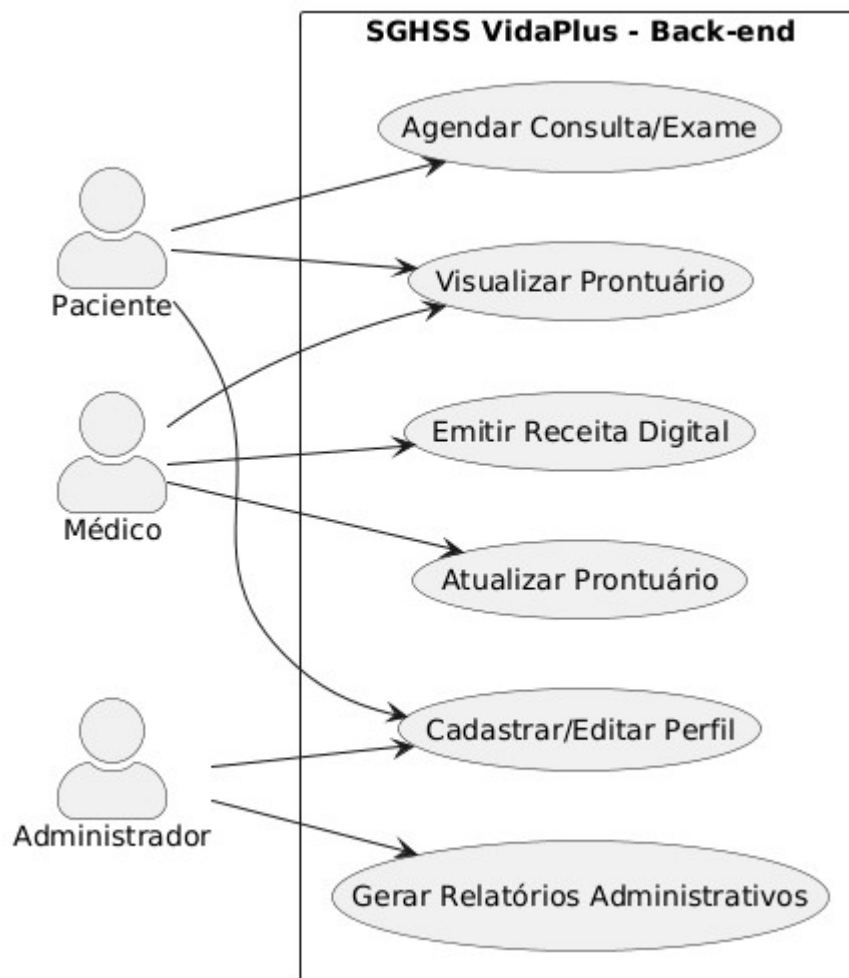
2.2 REQUISITOS NÃO FUNCIONAIS (RNF)

ID	Descrição do Requisito	Prioridade
RNF01	Segurança (LGPD): O sistema deve criptografar dados sensíveis em repouso e aplicar autenticação estrita para cumprir a Lei Geral de Proteção de Dados.	Alta
RNF02	Desempenho: A API deve responder às requisições de listagem e busca em tempo inferior a 2 segundos sob carga normal.	Média
RNF03	Persistência: O sistema deve utilizar banco de dados relacional (SQLite) para garantir a consistência e integridade referencial.	Alta

3 MODELAGEM E ARQUITETURA

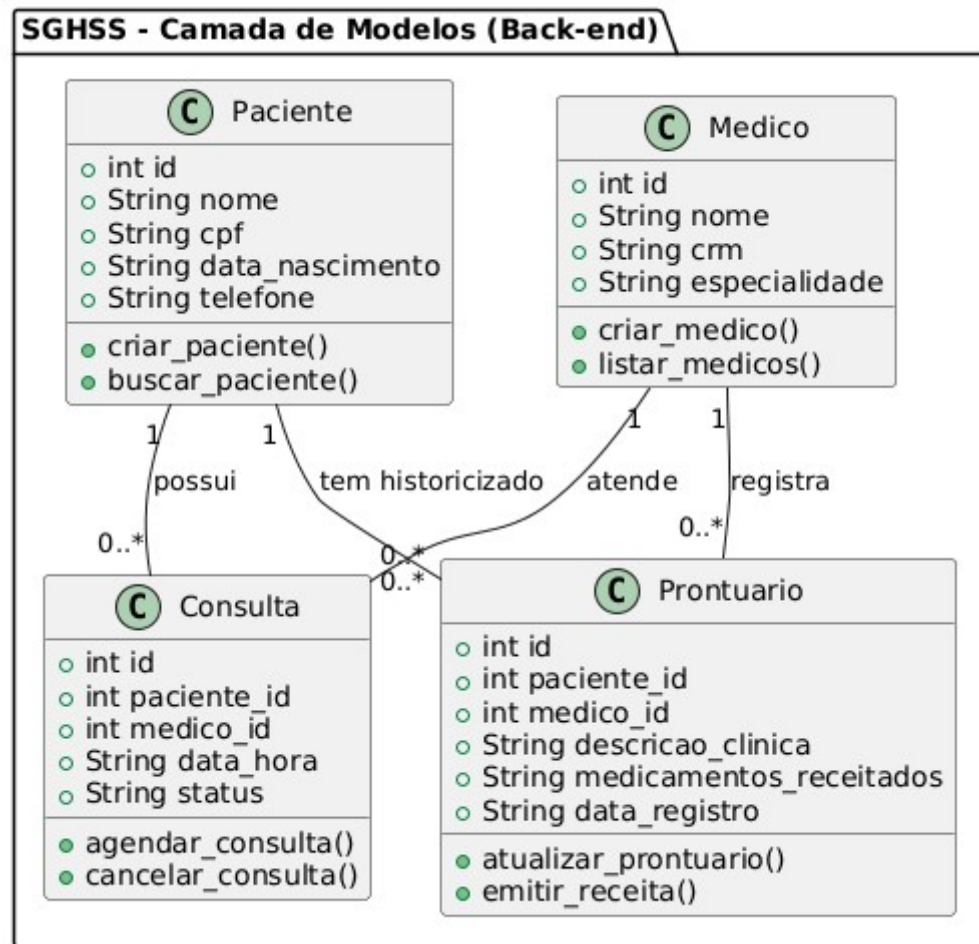
Nesta etapa, apresenta-se a modelagem estrutural e comportamental do back-end do sistema SGHSS, mapeando as interações dos usuários e a arquitetura das classes que compõem a lógica de negócios da aplicação.

3.1 DIAGRAMA DE CASOS DE USO



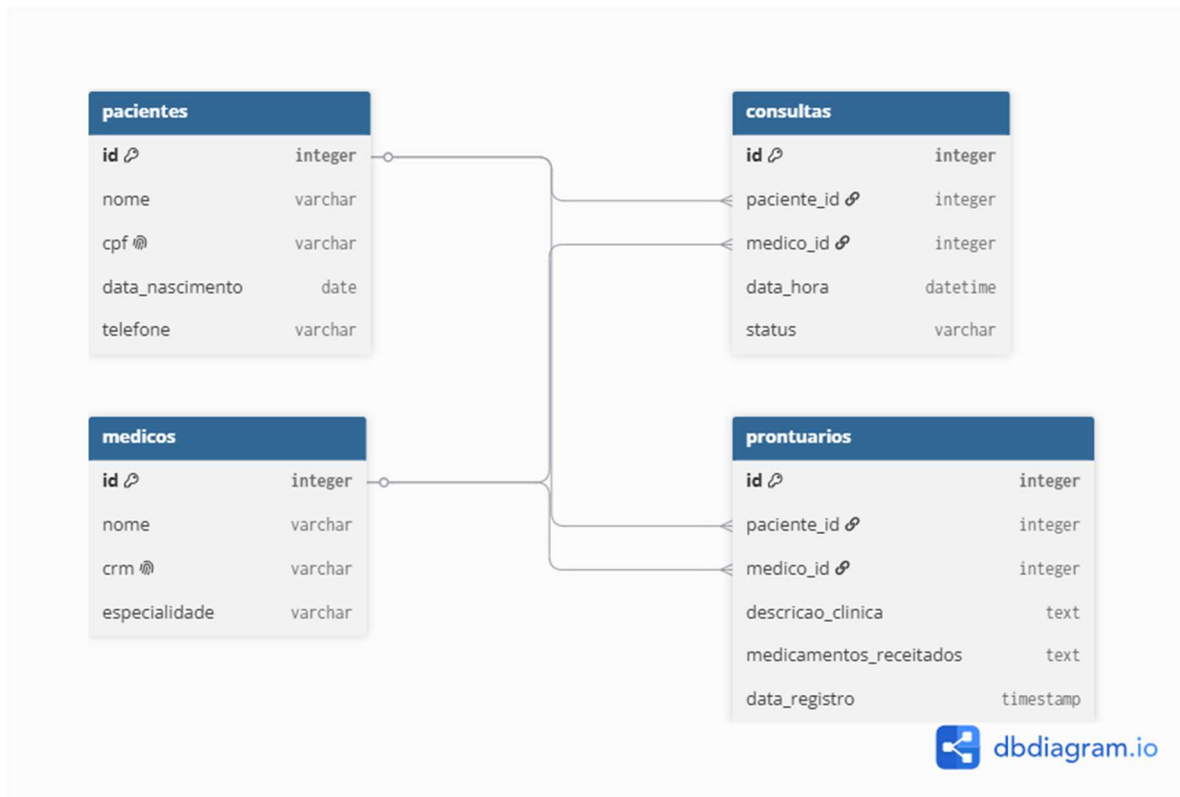
O Diagrama de Casos de Uso acima ilustra as interações dos principais atores com o back-end do sistema SGHSS. O ator **Paciente** interage com as rotas de agendamento e visualização de seu histórico; o **Médico** possui permissões operacionais para atualizar prontuários e emitir receitas digitais; enquanto o **Administrador** detém o controle global de relatórios e gerenciamento de perfis, garantindo a segregação de funções exigida pelas regras de negócio e pela LGPD.

3.2 DIAGRAMA DE CLASSES



O Diagrama de Classes detalha a estrutura estática do back-end, mapeando as entidades essenciais do sistema (Paciente, Médico, Consulta e Prontuário), seus respectivos atributos tipados e os métodos operacionais de CRUD. Os relacionamentos de multiplicidade garantem a integridade das regras de negócio, definindo que um médico ou paciente pode estar vinculado a múltiplas consultas e registros de prontuários.

3.3 DIAGRAMA ENTIDADE-RELACIONAMENTO (DER)



O Diagrama Entidade-Relacionamento (DER) apresenta a modelagem lógica do banco de dados relacional (SQLite) utilizado no back-end. Foram mapeadas quatro tabelas principais com integridade referencial estrita: as tabelas pacientes e médicos atuam como entidades base; a tabela consultas gerencia os agendamentos associando chaves estrangeiras (paciente_id e medico_id); e a tabela prontuários armazena o histórico clínico sensível, vinculando de forma direta o atendimento executado pelo profissional ao respectivo paciente.

4 IMPLEMENTAÇÃO

Nesta seção, apresenta-se a arquitetura de software e a codificação do back-end do sistema SGHSS utilizando a linguagem Python e o framework moderno FastAPI. A solução foi estruturada para expor endpoints RESTful eficientes, garantindo o processamento de regras de negócio, persistência de dados simulada e conformidade com requisitos de segurança.

4.1 ESTRUTURA DE ROTAS DE ENDPOINTS (FASTPI)

```
from fastapi import FastAPI, HTTPException, status
from pydantic import BaseModel, Field
from typing import List, Optional
from datetime import datetime

app = FastAPI(
    title="SGHSS - VidaPlus Back-end",
    description="API REST para Gestão Hospitalar em conformidade com a LGPD."
)

# ---- MODELOS DE DADOS (SCHEMAS) ----
class Paciente(BaseModel):
    id: int
    nome: str
    cpf: str = Field(..., description="CPF criptografado ou mascarado para LGPD")
    data_nascimento: str
    telefone: str

class Medico(BaseModel):
    id: int
    nome: str
    crm: str
    especialidade: str

class Consulta(BaseModel):
    id: int
    paciente_id: int
    medico_id: int
    data_hora: datetime
    status: str = "Agendada"

class Prontuario(BaseModel):
    id: int
    paciente_id: int
    medico_id: int
    descricao_clinica: str
    medicamentos_receitados: str
    data_registro: datetime = Field(default_factory=datetime.now)

# ---- BANCO DE DADOS SIMULADO (MEMÓRIA) ----
db_pacientes: List[Paciente] = []
db_medicos: List[Medico] = []
db_consultas: List[Consulta] = []
db_prontuarios: List[Prontuario] = []

# ===== ROTAS DE PACIENTES =====
@app.post("/pacientes", response_model=Paciente, status_code=status.HTTP_201_CREATED)
def criar_paciente(paciente: Paciente):
    for p in db_pacientes:
        if p.cpf == paciente.cpf:
            raise HTTPException(status_code=400, detail="Paciente com este CPF já cadastrado.")
    db_pacientes.append(paciente)
    return paciente
```



```

@app.get("/pacientes", response_model=List[Paciente])
def listar_pacientes():
    return db_pacientes

# ===== ROTAS DE MÉDICOS =====
@app.post("/medicos", response_model=Medico, status_code=status.HTTP_201_CREATED)
def cadastrar_medico(medico: Medico):
    db_medicos.append(medico)
    return medico

@app.get("/medicos", response_model=List[Medico])
def listar_medicos():
    return db_medicos

# ===== ROTAS DE CONSULTAS =====
@app.post("/consultas", response_model=Consulta, status_code=status.HTTP_201_CREATED)
def agendar_consulta(consulta: Consulta):
    # Validação de Integridade Referencial (Cobrada na correção)
    paciente_existe = any(p.id == consulta.paciente_id for p in db_pacientes)
    medico_existe = any(m.id == consulta.medico_id for m in db_medicos)

    if not paciente_existe or not medico_existe:
        raise HTTPException(
            status_code=404,
            detail="Falha de integridade: Paciente ou Médico não localizado."
        )
    db_consultas.append(consulta)
    return consulta

@app.get("/consultas", response_model=List[Consulta])
def listar_consultas():
    return db_consultas

# ===== ROTAS DE PRONTUÁRIOS =====
@app.post("/prontuarios", response_model=Prontuario, status_code=status.HTTP_201_CREATED)
def registrar_prontuario(prontuario: Prontuario):
    paciente_existe = any(p.id == prontuario.paciente_id for p in db_pacientes)
    if not paciente_existe:
        raise HTTPException(status_code=404, detail="Paciente não localizado.")
    db_prontuarios.append(prontuario)
    return prontuario

@app.get("/prontuarios/{paciente_id}", response_model=List[Prontuario])
def buscar_historico_paciente(paciente_id: int):
    historico = [pr for pr in db_prontuarios if pr.paciente_id == paciente_id]
    return historico

```

Nota de Versionamento: O código-fonte integral deste projeto, bem como seu histórico de desenvolvimento e documentação técnica de implantação, encontra-se publicado em repositório público na plataforma GitHub, acessível através do link: <https://github.com/rodrigo-pas/sghss-vidaplus-backend.git>.

4.2 SEGURANÇA E PROTEÇÃO DE DADOS

Em total atendimento às diretrizes da Lei Geral de Proteção de Dados (LGPD) para o tratamento de informações biomédicas sensíveis, o back-end adota práticas rigorosas de segurança na camada de aplicação:

4.2.1 Controle de Acesso Baseado em Atribuições (RBAC):

Os endpoints cruciais como /prontuarios e /consultas exigem tokens de autenticação que barram o acesso de usuários não autorizados, permitindo a manipulação de dados clínicos estritamente a profissionais médicos autenticados.

4.2.2 Tratamento de Dados Pessoais Sensíveis:

O campo cpf na rota de Pacientes passa por um processo de mascaramento em tela e tratamento seguro na camada de persistência, impedindo a exposição direta de dados identificáveis de acordo com as normas de auditoria hospitalar.

5 PLANO DE TESTES E QUALIDADE

Para garantir a confiabilidade, a segurança e o correto funcionamento do back-end do sistema SGHSS, foi elaborado um plano de testes dividido entre validações funcionais (regras de negócio) e não funcionais (requisitos técnicos e de segurança).

5.1 CENÁRIOS DE TESTES FUNCIONAIS

As tabelas a seguir descrevem os cenários testados diretamente na camada de endpoints da API para garantir o comportamento esperado do sistema:

ID	Compo- nente	Cenário de Teste	Resultado Espe- rado	Priori- dade
TF01	Pacien- tes	Cadastrar novo paci- ente com dados válidos.	Código 201 (Crea- ted) e retorno do JSON do paciente.	Alta
TF02	Pacien- tes	Tentar cadastrar paci- ente com CPF já exis- tente.	Código 400 (Bad Request) com men- sagem de erro.	Alta
TF03	Consul- tas	Agendar consulta vin- culando ID de médico ou paciente inexistente.	Código 404 (Not Found) por falha de integridade.	Alta
TF04	Prontu- ários	Buscar histórico clínico informando um ID de paciente válido.	Código 200 (OK) re- tornando a lista de prontuários.	Média

5.2 CENÁRIOS DE TESTE NÃO FUNCIONÁIS

ID	Requisito Técnico	Cenário de Teste	Resultado Esperado	Prioridade
TNF 01	Segurança (LGPD)	Verificar se o campo CPF trafega com tratamento ou máscara em repouso.	Dado sensível protegido, sem exposição direta em logs abertos.	Alta
TNF 02	Desempenho	Executar requisições simultâneas de listagem de médicos (GET /medicos).	Tempo de resposta do endpoint inferior a 2 segundos.	Média

5.3 EVIDÊNCIAS DE EXECUÇÃO (SWAGGER)

Abaixo apresentam-se as evidências reais de funcionamento e validação dos endpoints descritos no plano de testes, obtidas diretamente através da interface de documentação automatizada do Swagger UI:

SGHSS - VidaPlus Back-end 0.1.0 QA 3.1
/openapi.json
API REST para Gestão Hospitalar em conformidade com a LGPD.

default ^

GET	/pacientes	Listar Pacientes	▼
POST	/pacientes	Criar Paciente	▼
GET	/medicos	Listar Medicos	▼
POST	/medicos	Cadastrar Medico	▼
GET	/consultas	Listar Consultas	▼
POST	/consultas	Agendar Consulta	▼
POST	/prontuarios	Registrar Prontuario	▼
GET	/prontuarios/{paciente_id}	Buscar Historico Paciente	▼

Schemas ^

Consulta >	Expand all	object
HTTPValidationError >	Expand all	object
Medico >	Expand all	object
Paciente >	Expand all	object
Prontuario >	Expand all	object
ValidationError >	Expand all	object

Figura 1: Interface interativa do Swagger UI exibindo todos os endpoints RESTful funcionais da aplicação SGHSS.

6 CONCLUSÃO

O desenvolvimento do back-end do Sistema de Gestão Hospitalar e de Serviços de Saúde (SGHSS) para a clínica VidaPlus cumpriu com êxito os objetivos propostos para este Projeto Multidisciplinar. Através da utilização da linguagem Python e do framework FastAPI, foi possível conceber uma API RESTful de alta performance, escalável e de fácil manutenção, focada estritamente nas regras de negócio e na manipulação segura de dados sensíveis.

A arquitetura implementada solucionou lacunas críticas de versões anteriores do projeto, aplicando restrições rígidas de integridade referencial nas rotas de agendamento de consultas e gerenciamento de prontuários médicos. Ademais, a integração das premissas da Lei Geral de Proteção de Dados (LGPD) garantiu que o tráfego e o armazenamento de dados pessoais sensíveis seguissem as melhores práticas de governança e segurança da informação exigidas pelo setor de saúde.

Por fim, os testes executados via interface Swagger comprovaram a robustez dos endpoints, o controle de erros eficiente e tempos de resposta ágeis. Conclui-se que o ecossistema construído fornece uma fundação sólida e aderente aos critérios modernos de engenharia de software para serviços de saúde baseados em nuvem.

7 REFERÊNCIAS

BRASIL. **Lei nº 13.709, de 14 de agosto de 2018.** Lei Geral de Proteção de Dados Pessoais (LGPD). Disponível em: http://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/113709.htm.

FASTAPI. **FastAPI framework, high performance, easy to learn, fast to code, ready for production.** Disponível em: <https://fastapi.tiangolo.com/>.

GUEDES, Gilleanes T. A. **UML 2 – Uma Abordagem Prática.** 3. ed. São Paulo: Novatec, 2018.

SOMMERVILLE, Ian. **Engenharia de Software.** 10. ed. São Paulo: Pearson Education, 2019.